

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3047 – Administración y Mantenimiento de Sistemas

Ing. Vinicio Gabriel Paz Calderón



Proyecto 3 - Monitoreo ELK

José Pablo Orellana	21970
Diego Alberto Leiva	21572
Maria Marta Ramírez	21970
Gustavo Andrés González	21970
Renatto Esteban Guzmán	21646

Guatemala, 19 de noviembre de 2025

Introducción

Este documento describe el diseño, implementación y resultados del sistema de monitoreo desarrollado para el proyecto Jack's Cave API, como parte de la Fase 3 del curso de Administración de Sistemas.

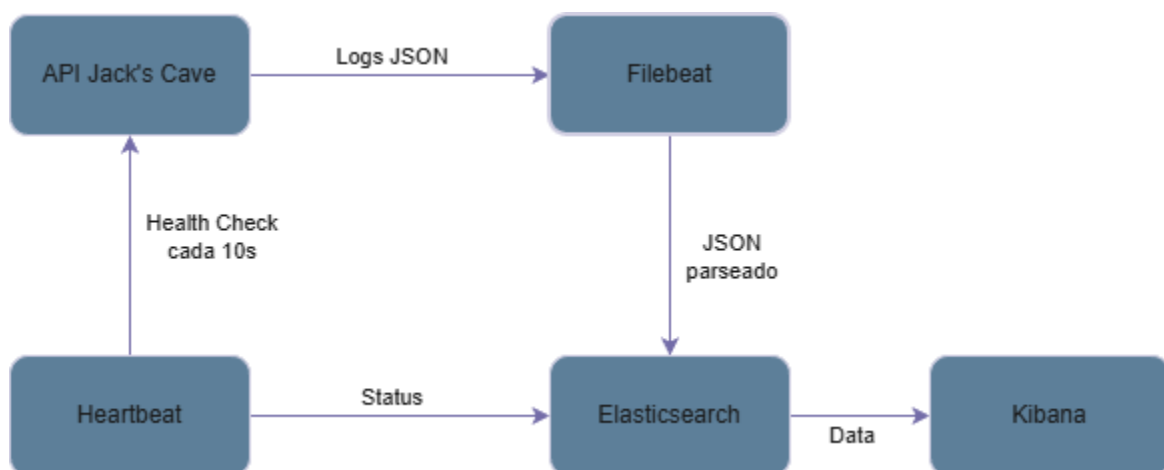
El objetivo fue construir un entorno de observabilidad utilizando el stack ELK (Elasticsearch, Logstash/Filebeat y Kibana), junto con Heartbeat, para monitorear la salud del sistema, la disponibilidad de la API, el tráfico de peticiones, errores y rendimiento en tiempo real.

Arquitectura General del Sistema

La solución se desplegó completamente en contenedores Docker, utilizando un archivo docker-compose.yml con los siguientes servicios:

Componente	Descripción
API (FastAPI)	API simulada representativa del proyecto real. Genera logs estructurados en formato JSON.
PostgreSQL	Base de datos utilizada por la API. Se configuró con healthcheck.
Elasticsearch 8.12	Motor de búsqueda donde se almacenan logs, métricas y datos del monitoreo.
Kibana 8.12	Interfaz para la visualización de datos.
Filebeat 8.12	Colector de logs, encargado de leer api.log y enviarlo a Elasticsearch.
Heartbeat 8.12	Monitor de uptime, verifica cada 10 segundos la disponibilidad de la API.

Diagrama de Arquitectura



Configuración Implementada

El archivo docker-compose.yml incluyó:

- Persistencia mediante volúmenes:
 - db_data (PostgreSQL)
 - es_data (Elasticsearch)
- Red compartida jacks-net
- Autoarranque y dependencias con depends_on
- Montaje de logs desde la API hacia Filebeat

El archivo filebeat.yml se configuró para:

- Leer logs desde /app/logs/api.log
- *Parsear* JSON automáticamente
- Enviar logs a Elasticsearch usando índice diario
- Añadir metadatos:
 - service.name: jacks-cave-api
 - env: dev

El archivo heartbeat.yml monitorea el endpoint /health cada 10 segundos genera un evento con:

- monitor.status: up o down
- http.response.status_code
- Latencia
- *Metadata* del monitor

La API genera logs JSON normalizados con los campos requeridos:

- @timestamp
- http.request.method
- http.response.status_code
- event.duration (ns)
- url.path
- service.name
- log.level (INFO/WARN/ERROR)

Estos logs son consumidos por Filebeat y almacenados en Elasticsearch.

Resultados Obtenidos

- La arquitectura se despliega correctamente vía Docker Compose, manteniendo persistencia entre sesiones gracias al volumen es_data.
- Filebeat ingiere y procesa logs JSON sin errores.
- Heartbeat detecta correctamente la disponibilidad del servicio, registrando eventos UP/DOWN en Elasticsearch y evidenciando el estado en el Dashboard
- El dashboard muestra métricas correctas de tráfico, latencia, errores y disponibilidad.
- Se comprobó el funcionamiento simulando fallas reales, como errores 400 y 404, requests incorrectos, apagado del contenedor API (DOWN detectado)
- La persistencia de Elasticsearch garantiza que el dashboard no se pierde tras reinicios.

Conclusiones

- Se logró implementar un sistema de observabilidad básico pero funcional y completo.
- El stack ELK permitió centralizar logs, métricas y visualizaciones de forma profesional.
- Filebeat y Heartbeat brindaron monitoreo real de trazas y disponibilidad.
- Kibana permitió construir un dashboard claro y útil para personal de soporte.